

Создание Окна Авторизации

**КОГДА СОЗДАЕТЕ ПРОЕКТ ПРОВЕРЬТЕ, ЧТОБЫ *НЕ* СТОЯЛА
ГАЛОЧКА «ПОМЕСТИТЬ РЕШЕНИЕ И ПРОЕКТ В ОДНОМ
КАТАЛОГЕ» ИНАЧЕ КОМАНДЫ ENTITY FRAMEWORK *НЕ*
СРАБОТАЮТ**



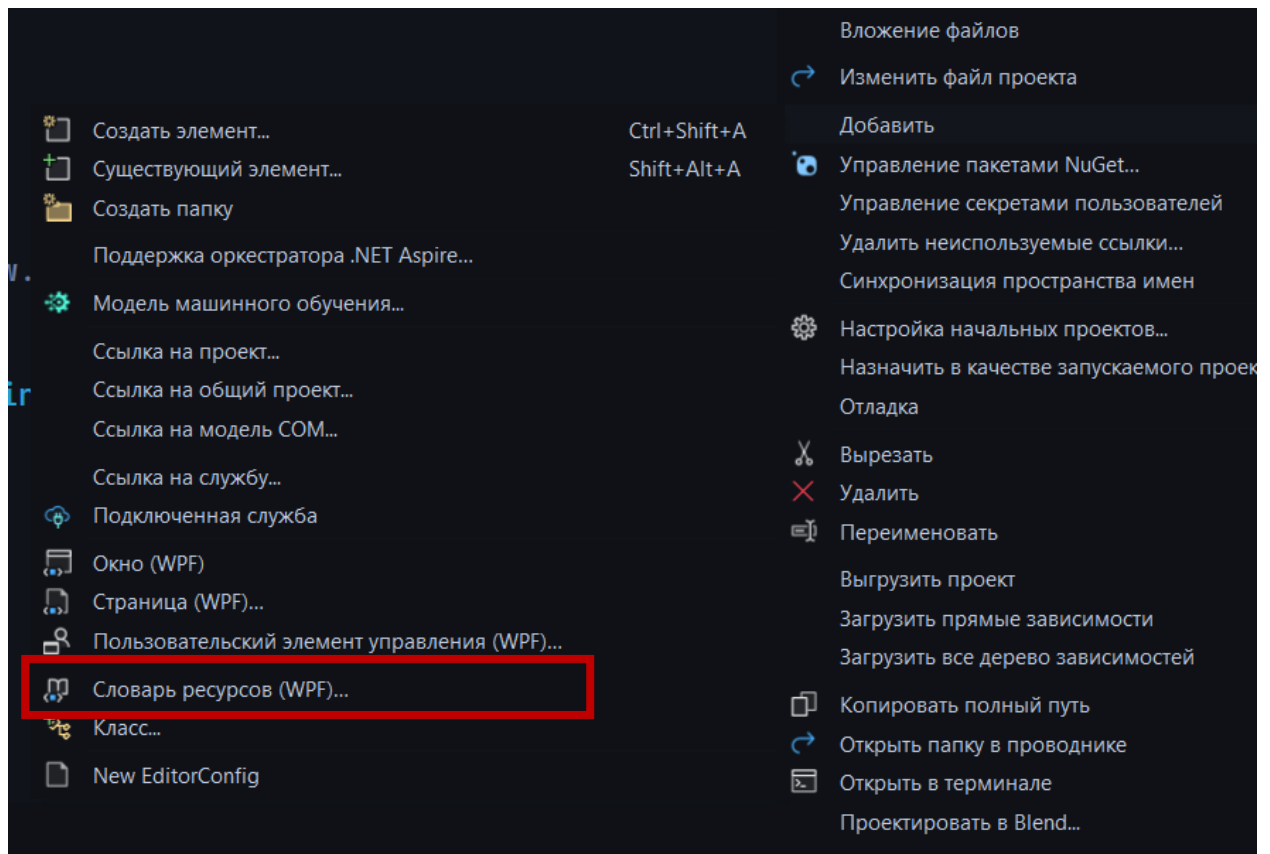
1. Добавьте окно для авторизации в проект (или переименуйте MainWindow).
2. Добавьте необходимые пакеты EntityFramework и подключите базу данных. (Файл EF подключение и файл команды для powershell)

РАЗМЕТКА ОКНА

Для удобства реализации рекомендуется прописать стили для кнопок и шрифта.

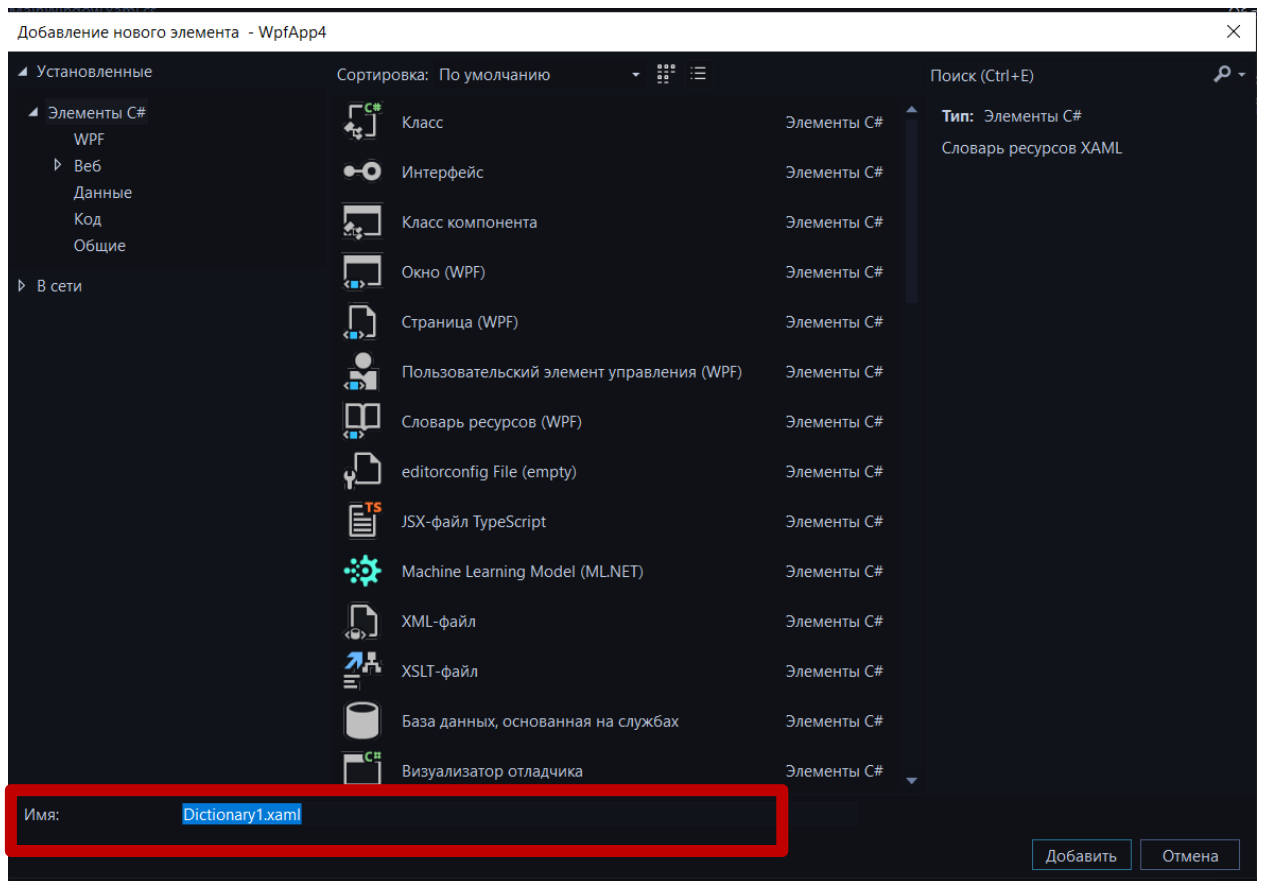
ДОБАВЛЕНИЕ СТИЛЕЙ

Нужно создать словарь ресурсов и ссылаться на него в App.xaml
Для этого ПКМ по проекту → «Добавить» → «Словарь ресурсов»



В появившемся окне пропишите название словаря

Можно переписать полностью название без расширения, VS сам допишет его. **(НЕ ОСТАВЛЯЙТЕ Dictionary1 И ТД, ДАВАЙТЕ ПРАВИЛЬНЫЕ НАЗВАНИЯ ДЛЯ ФАЙЛОВ И КЛАССОВ)**

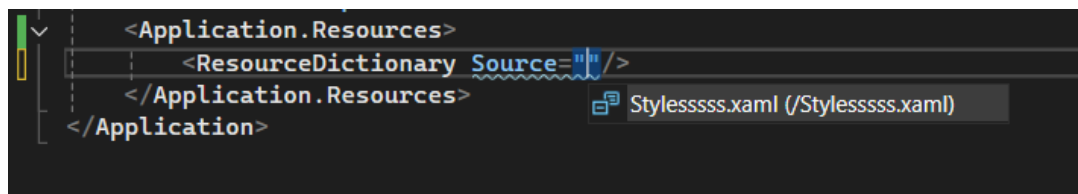


В App.xaml пропишем

```
<Application.Resources>
  <ResourceDictionary Source="/Stylesssss.xaml"/>
</Application.Resources>
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

Visual Studio предложит выбрать словарь ресурсов

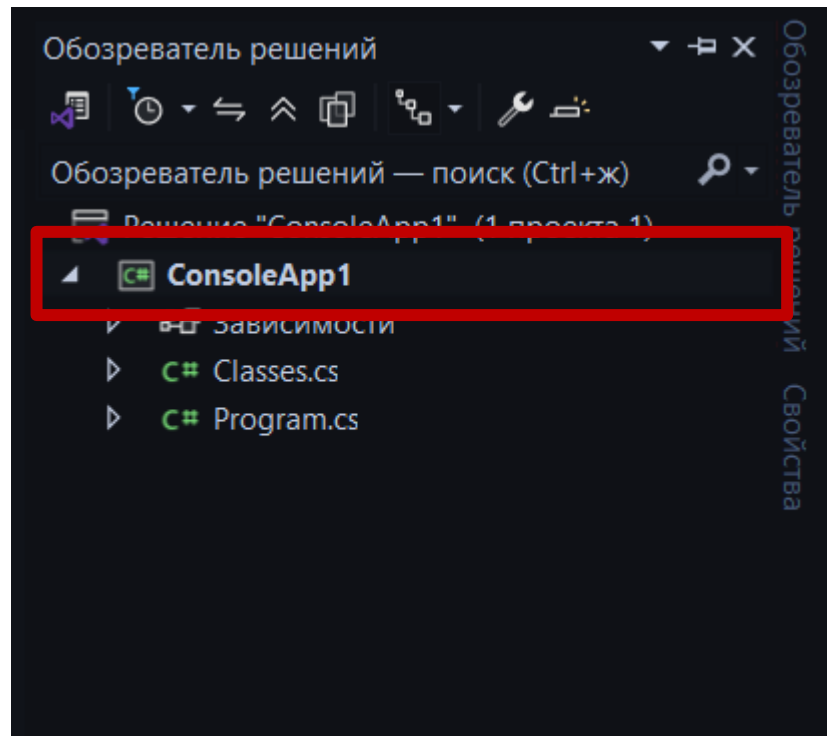


Для применения стиля ко всем элементам не прописывайте ключ (x:Key)

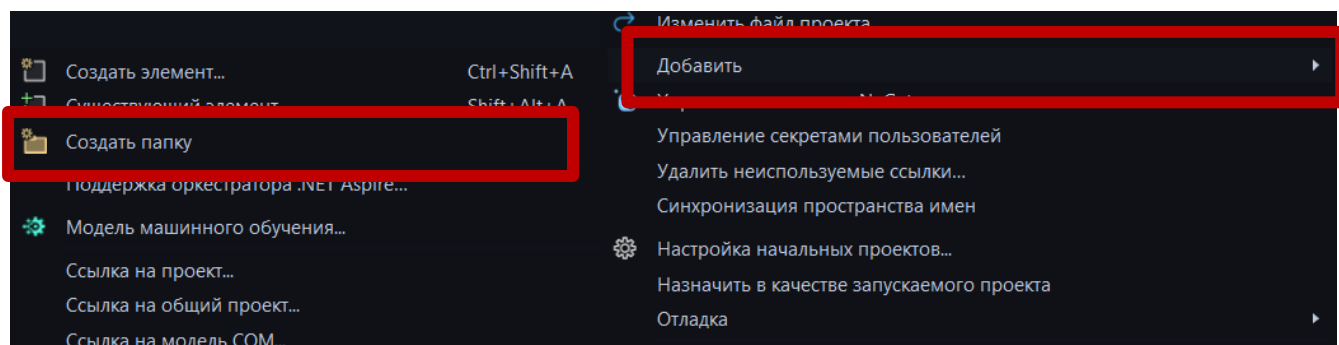
1. Измените название окна на «Авторизация», установите его размеры, иконку, и разместите начальную позицию (WindowStartupLocation) в центре, а также ограничьте изменение размеров окна (ResizeMode).

2. Добавьте необходимые изображения для иконки и логотипа, можно также сразу импортировать изображения товаров

2.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по проекту)

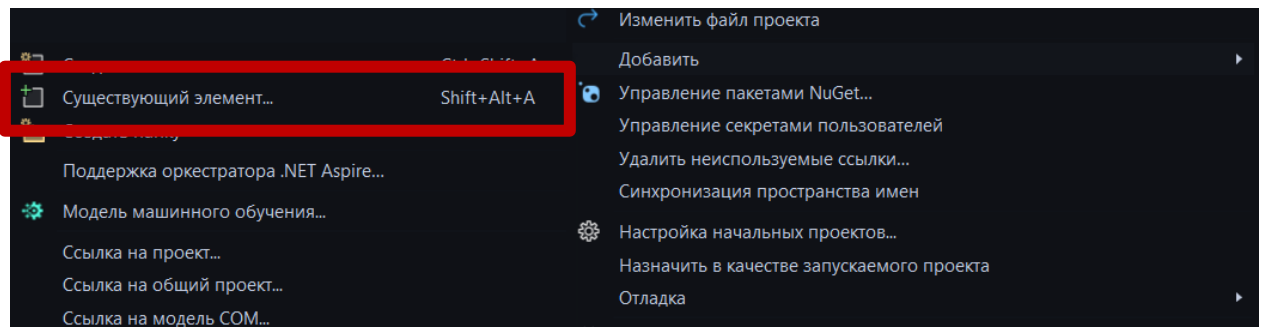


Создайте папку Images или Resources, для этого выберите «Добавить» → «Папку»

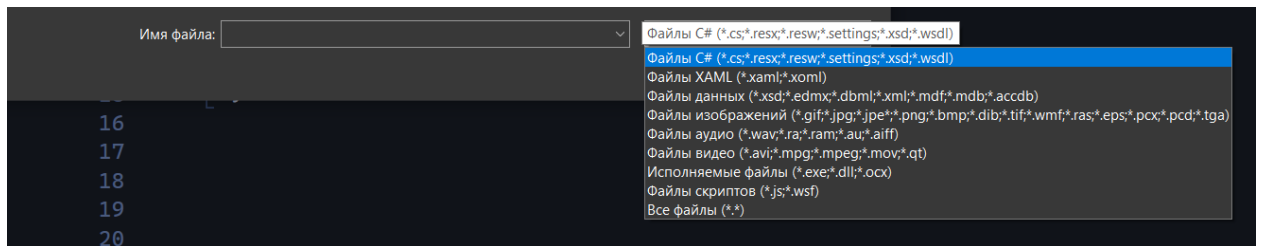


2.2 Поместите туда все необходимые изображения

Так же кликните ПКМ по проекту → Добавить → Существующий элемент

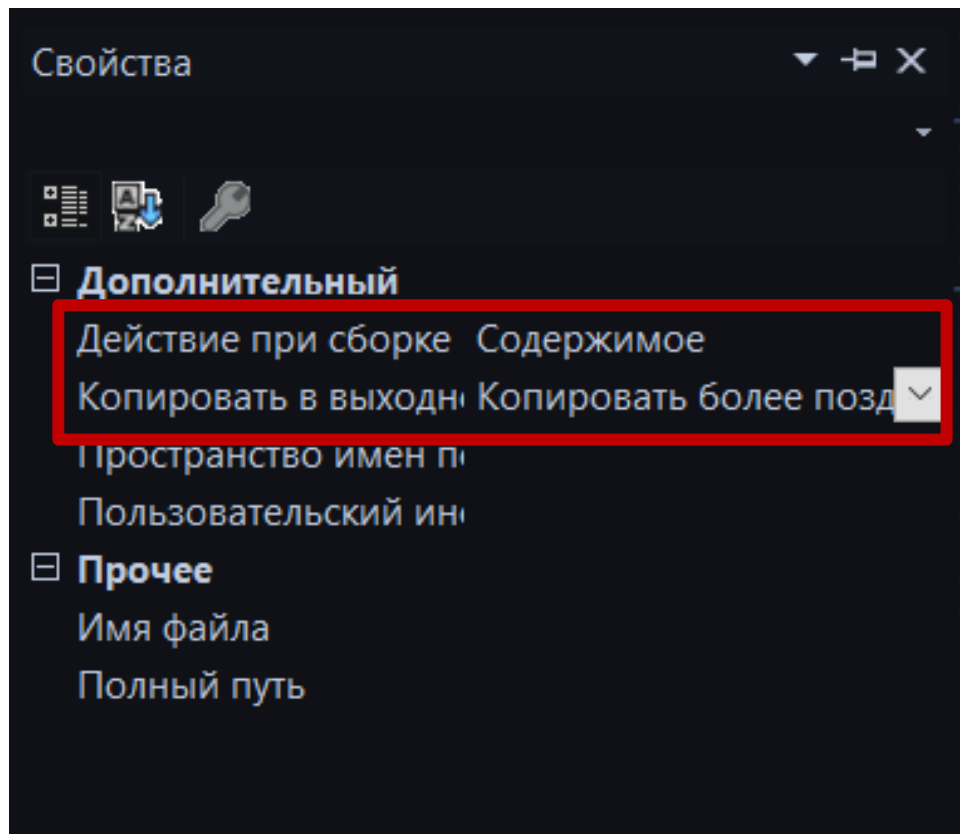


В диалоговом окне установите фильтр «Все файлы» и добавьте необходимые изображения (можно сразу несколько)



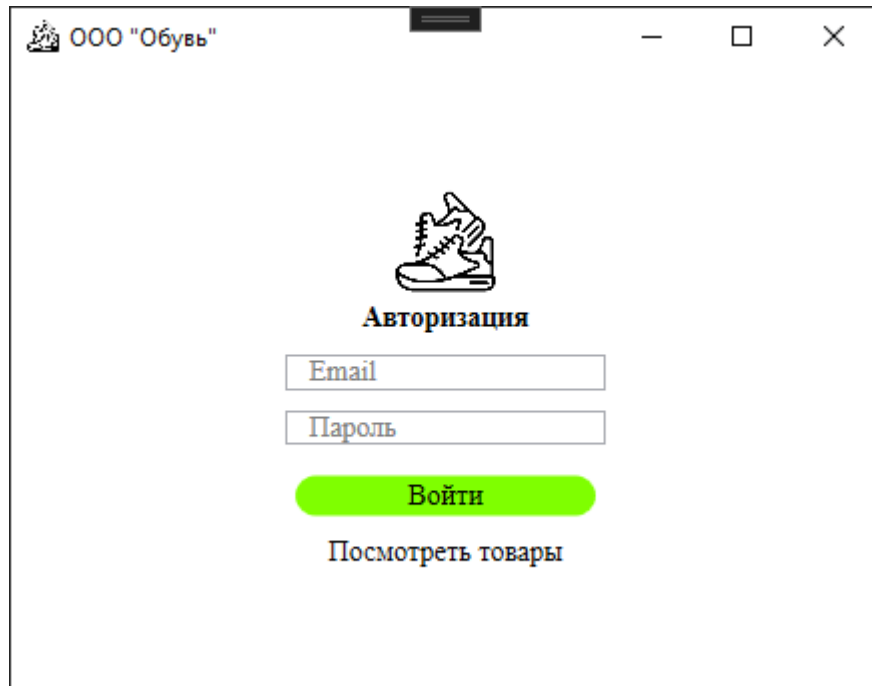
2.3 Измените свойства изображений. Для этого выделите несколько изображений (через Shift или Ctrl), перейдите в свойства (если этого окна нет, то в верхней панели нажмите Вид → Свойства или F4).

Установите свойство «Действие при сборке» → «Содержимое», «Копировать в выходной каталог» → «Копировать более позднюю версию»



3. Grid можно заменить на StackPanel, чтобы элементы шли последовательно друг за другом.
4. Добавьте логотип компании
5. Разместите надпись «Авторизация»
6. Разместите 2 текстовых поля для ввода (не забудьте подписать, где какое поле), для пароля лучше использовать PasswordBox
7. Добавьте кнопку для входа
8. Добавьте текстовый блок или кнопку, чтобы можно было просмотреть товары без авторизации в роли гостя

!!!ПРИМЕР!!!



ООО "Обувь"



Авторизация

Email

Пароль

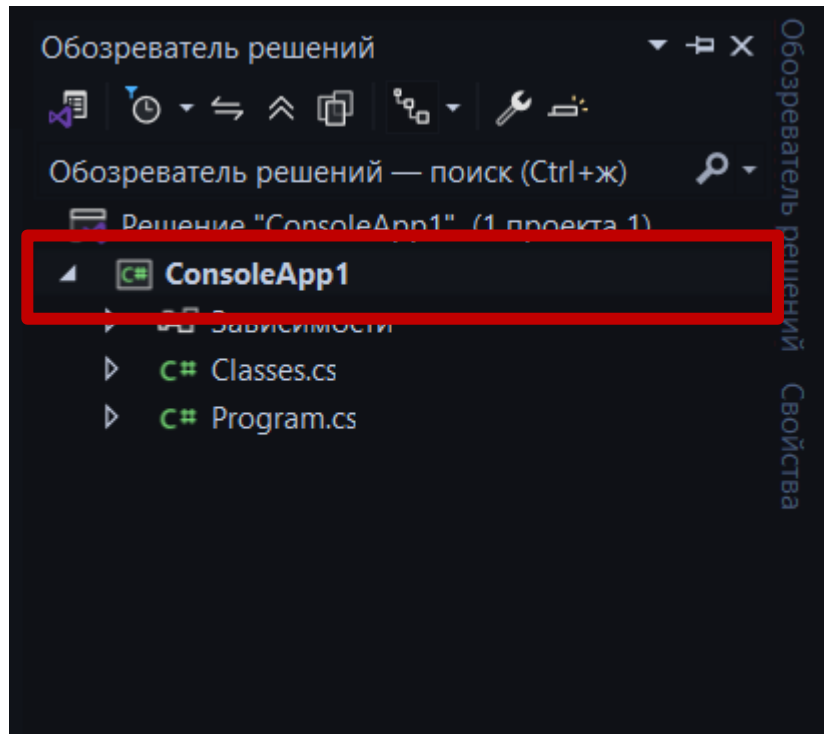
Войти

[Посмотреть товары](#)

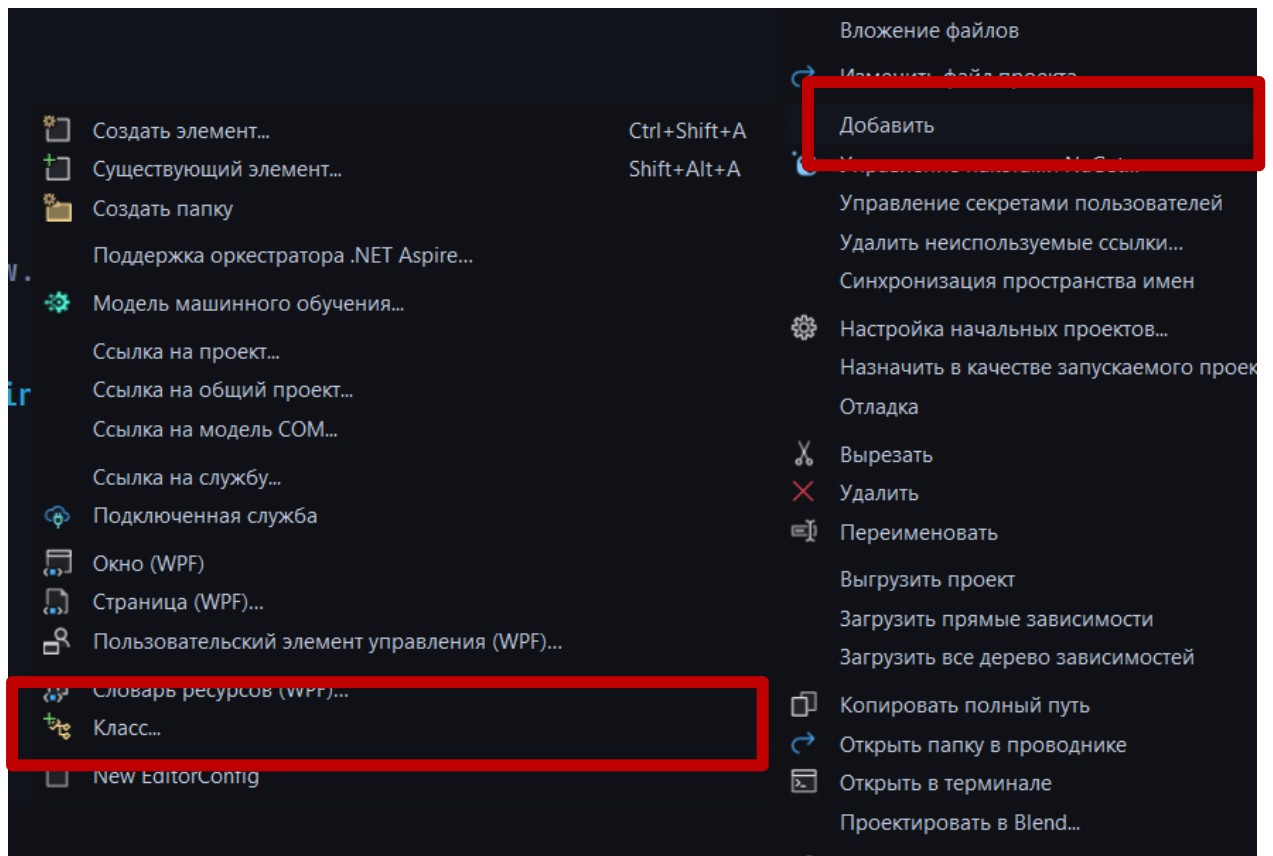
(Не обязательно делать как на картинке, то есть кнопку закруглять НЕ нужно, размешать текстовые поля внутри TextBox тоже НЕ нужно, проще будет реализовать последовательность TextBlock → TextBox)

ЛОГИКА ОКНА

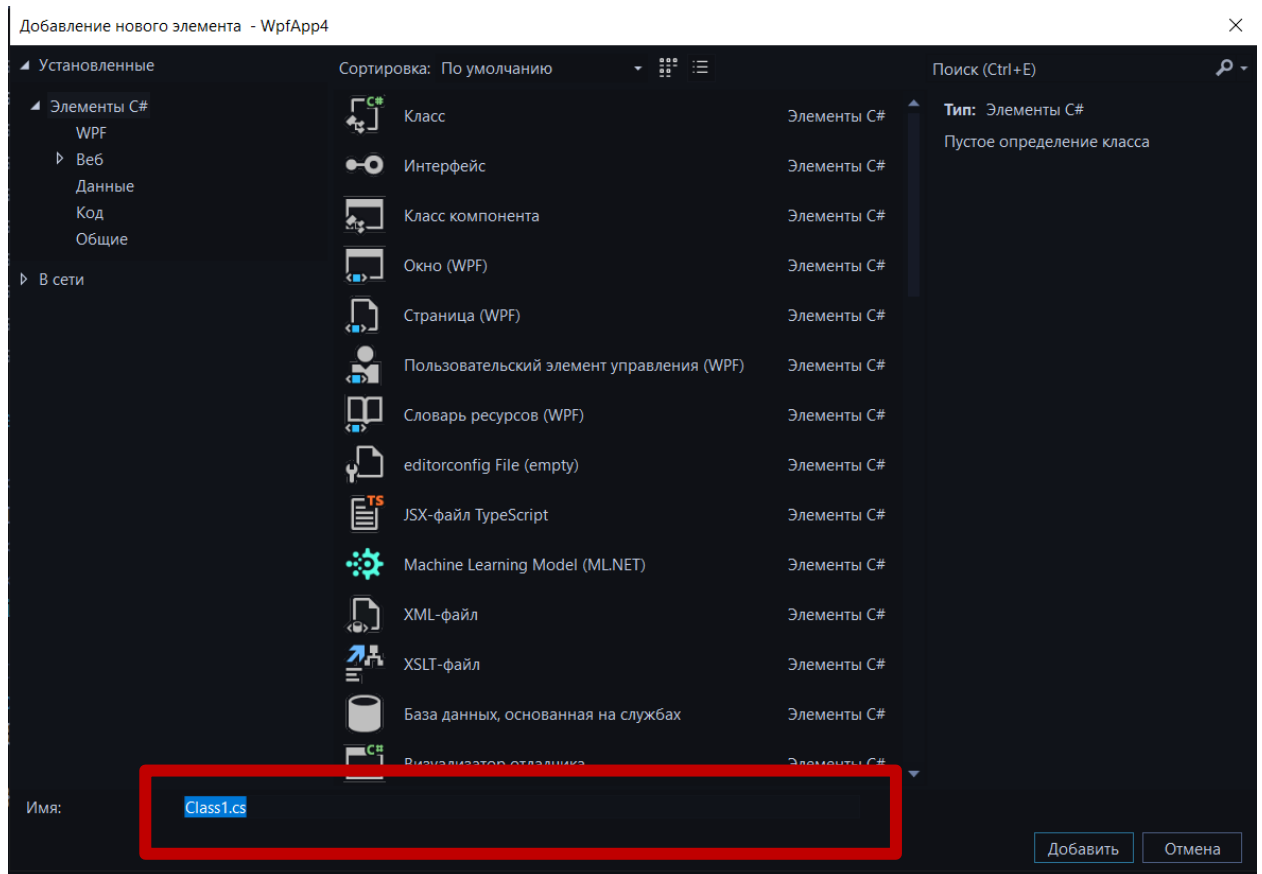
1. Откройте C#-файл (расширение *.xaml.cs) окна авторизации.
2. Для реализации распределения ролей создайте static класс (отдельным файлом) `CurrentUser`
 - 2.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по проекту)



- 2.2 Выберите «Добавить» → «Класс»



2.3 В появившемся окне присвойте название классу (**НЕ ОСТАВЛЯЙТЕ CLASS1 И ТД, ДАВАЙТЕ ПРАВИЛЬНЫЕ НАЗВАНИЯ ДЛЯ ФАЙЛОВ И КЛАССОВ**)



Можно переписать полностью название без расширения, VS сам допишет его. **ПОМНИТЕ**, что для классов используется соглашение об именовании **PascalCase**, это значит, что *каждое слово начинается с заглавной (большой) буквы*.

2.4 Поменяйте модификатор доступа **internal** на **public**, чтобы класс был доступен и в других классах (файлах, окнах).

```
namespace WpfApp4
{
    Ссылка: 0
    internal class CurrentUser
    {
    }
}
```



```
namespace WpfApp4
{
    Ссылка: 0
    public class CurrentUser
    {
    }
}
```

Не забудьте добавить ключевое слово **static**, чтобы сделать класс статическим (не требуют создания объекта и позволяют обращаться к полям и методам)

```
namespace WpfApp4
{
    Ссылка: 0
    public static class CurrentUser
    {
    }
}
```

В классе пропишите статические поля

```
public static int Id { get; set; }
public static string FullName { get; set; } = "Гость";
public static string Role { get; set; } = "Гость";
```

3. Вернемся в окно авторизации. Для удобства можно создать отдельный метод для перехода на окно с товарами

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
private void OpenProdWin()
{
    CatalogWindow mn = new(); // создание объекта окна
    mn.Show();                // открытие окна
    Close();                   // закрытие текущего окна
}
```

4. Для элемента просмотра каталога без авторизации пропишите обработчик событий (Click для кнопки и MouseLeftButtonUp для текстового блока). В нем установите Id = 0 и все остальные поля на «Гость». После вызовите метод открытия окна с каталогом

5. Теперь реализуем кнопку авторизации

В обработчике событий для кнопки входа создадим переменные и присвоим им данные, введенные в поля для ввода, для этого необходимо использовать:

```
Текстовое_поле.Text.Trim();
```

Text — считывает текст из поля;

Trim() — является необязательным элементом, он просто убирает пробелы в конце.

Для **PasswordBox**:

```
Текстовое_поле.Password.Trim();
```

Не забудьте обработать исключение и вывести сообщение для пользователя, если строки будут пустыми.

(string.IsNullOrEmpty(переменная))

Далее нам нужно использовать подключение к БД:

```
using var db = new контекстБД();
```

using откроет и закроет подключение к БД.

Создаем пользователя и считываем из БД роль, почту и пароль, используйте **LINQ** – запросы:

```
var user = db.Users.Include(u => u.Role)
.FirstOrDefault(u => u.Email == email && u.Passw == passw);
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СВОИ ТАБЛИЦЫ

Здесь мы с помощью Include используем связанную таблицу Ролей, а с помощью FirstOrDefault выбираем первое совпадение с логином и паролем.

Обработайте исключение, если логин и пароль не совпадет.
FirsOrDefault возвращает null, если совпадения не найдены.

Далее CurrentUser присвойте айди, имя и роль.

Откройте окно каталога.

6. Создадим метод для входа в роли гостя.

Для кнопки или текста создайте обработчик событий. Присвойте текущему пользователю айди = 0, роль и имя: гость. Откройте окно с каталогом (для удобства можно вынести в отдельный метод и вызывать его)